

Lab1

Question 1

Q1. DAXPY Loop: D stands for Double precision, A is a scalar value, X and Y are one-dimensional vectors of size 2^{16} each, P stands for Plus. The operation to be completed in one iteration is $X[i] = a \cdot X[i] + Y[i]$. Your task is to compare the speedup (in execution time) gained by increasing the number of threads. Start from a 2 thread implementation. How many threads give the max speedup? What happens if no. of threads are increased beyond this point? Why?

Sequential Code

```
1  #include <iostream>
2  #include <omp.h>
3  using namespace std;
4  #define N 65536 // 2^16
5  double X[N], Y[N];
6  void init() {
7      for(int i = 0; i < N; i++) {
8          X[i] = i + 1;
9          Y[i] = i + 2;
10     }
11 }
12 int main() {
13     double a = 2.5;
14     double t1, t2;
15     cout << "DAXPY: X[i] = a*X[i] + Y[i]" << endl;
16     cout << "Size = " << N << endl << endl;
17     init();
18     t1 = omp_get_wtime();
19     for(int k = 0; k < 1000; k++) {
20         init();
21         for(int i = 0; i < N; i++) {
22             X[i] = a * X[i] + Y[i];
23         }
24     }
25     t2 = omp_get_wtime();
26     cout << "Serial time: " << (t2 - t1) << " sec" << endl;
27     return 0;
28 }
```

Sequential Output:

```
1 DAXPY: X[i] = a*X[i] + Y[i]
2 Size = 65536
3 Serial time: 0.343 sec
```

Parallel Code

```
1  #include <iostream>
2  #include <omp.h>
3  using namespace std;
4  #define N 65536 // 2^16
5  double X[N], Y[N];
```

```

6 void init() {
7     for(int i = 0; i < N; i++) {
8         X[i] = i + 1;
9         Y[i] = i + 2;
10    }
11 }
12 int main() {
13     double a = 2.5;
14     double t1, t2;
15     int threads[] = {1, 2, 4, 8, 16};
16     cout << "DAXPY: X[i] = a*X[i] + Y[i]" << endl;
17     cout << "Size = " << N << endl << endl;
18     // serial version for comparison
19     init();
20     t1 = omp_get_wtime();
21     for(int k = 0; k < 1000; k++) {
22         init();
23         for(int i = 0; i < N; i++) {
24             X[i] = a * X[i] + Y[i];
25         }
26     }
27     t2 = omp_get_wtime();
28     double serial_time = t2 - t1;
29     cout << "Serial time: " << serial_time << " sec" << endl << endl;
30     // parallel versions
31     cout << "Threads\tTime\t\tSpeedup" << endl;
32     cout << "-----" << endl;
33     for(int j = 0; j < 5; j++) {
34         int nthreads = threads[j];
35         omp_set_num_threads(nthreads);
36         init();
37         t1 = omp_get_wtime();
38         for(int k = 0; k < 1000; k++) {
39             init();
40             #pragma omp parallel for
41             for(int i = 0; i < N; i++) {
42                 X[i] = a * X[i] + Y[i];
43             }
44         }
45         t2 = omp_get_wtime();
46         double ptime = t2 - t1;
47         double speedup = serial_time / ptime;
48         cout << nthreads << "\t" << ptime << "\t" << speedup << "x" << endl;
49     }
50     return 0;
51 }

```

Parallel Output:

```

1 DAXPY: X[i] = a*X[i] + Y[i]
2 Size = 65536
3 Serial time: 0.343 sec
4 Threads Time           Speedup
5 -----
6
7 1      0.356           0.963x
8
9 2      0.333           1.030x
10

```

11	4	0.306	1.121x
12			
13	8	0.291	1.179x
14			
15	16	0.389	0.882x
16	Best speedup: 1.179x with 8 threads		

Answer

1. **Speedup increases with threads up to a point:** The speedup increases from 1 thread to 8 threads, reaching a maximum of approximately 1.18x with 8 threads.
2. **Diminishing returns after optimal threads:** Beyond 8 threads, the speedup actually decreases. With 16 threads, the speedup drops to 0.88x which is slower than serial.
3. **Limited speedup due to memory-bound nature:** DAXPY is a memory-bound operation. The computation is simple (multiply-add) but requires reading two arrays and writing to one. Memory bandwidth becomes the bottleneck, not CPU cycles.
4. **Thread overhead:** Creating and managing threads has overhead. For small workloads or memory-bound operations, this overhead can outweigh the benefits of parallelization.
5. **Amdahl's Law:** The speedup is limited by the serial portion of the code (initialization) and the memory access patterns.

Conclusion: For memory-bound operations like DAXPY, we cannot achieve linear speedup. The optimal number of threads depends on the memory bandwidth and cache behavior of the system.

Question 2

Q2. Matrix Multiply : Build a parallel implementation of multiplication of large matrices (Eg. size could be 1000x1000). Repeat the experiment from the previous question for this implementation. Think about how to partition the work amongst the threads – which elements of the product array will be calculated by each thread? Implement 2 version of parallel implementation for given task. First, use 1D threading(i.e., make a single loop run in parallel). Second, use 2D threading (i.e., use nested looping to the most suitable loops to be parallelized)

Sequential Code

```
1  #include <iostream>
2  #include <omp.h>
3  #include <cstdlib>
4  using namespace std;
5  #define N 1000
6
7  double A[N][N], B[N][N], C[N][N];
8  void init_matrix() {
9      for(int i = 0; i < N; i++) {
10         for(int j = 0; j < N; j++) {
11             A[i][j] = rand() % 100 / 100.0;
12             B[i][j] = rand() % 100 / 100.0;
13             C[i][j] = 0;
14         }
15     }
16 }
17
```

```

18 int main() {
19     double t1, t2;
20     srand(42);
21     init_matrix();
22     cout << "Matrix Multiplication " << N << "x" << N << endl;
23     // serial
24     t1 = omp_get_wtime();
25     for(int i = 0; i < N; i++) {
26         for(int j = 0; j < N; j++) {
27             C[i][j] = 0;
28             for(int k = 0; k < N; k++) {
29                 C[i][j] += A[i][k] * B[k][j];
30             }
31         }
32     }
33     t2 = omp_get_wtime();
34     cout << "Serial time: " << (t2 - t1) << " sec" << endl;
35     return 0;
36 }

```

Sequential Output:

```

1 Matrix Multiplication 1000x1000
2 Running serial...
3 Serial time: 6.359 sec

```

Parallel Code (1D Threading)

```

1 #include <iostream>
2 #include <omp.h>
3 #include <cstdlib>
4 using namespace std;
5 #define N 1000
6
7 double A[N][N], B[N][N], C[N][N];
8
9 int main() {
10     double t1, t2;
11     int threads[] = {1, 2, 4, 8, 16};
12     srand(42);
13     // initialize matrices...
14     // 1D parallel (outer loop only)
15     for(int j = 0; j < 5; j++) {
16         int nt = threads[j];
17         omp_set_num_threads(nt);
18         t1 = omp_get_wtime();
19         #pragma omp parallel for
20         for(int i = 0; i < N; i++) {
21             for(int j = 0; j < N; j++) {
22                 C[i][j] = 0;
23                 for(int k = 0; k < N; k++) {
24                     C[i][j] += A[i][k] * B[k][j];
25                 }
26             }
27         }
28         t2 = omp_get_wtime();
29         cout << nt << " threads: " << (t2-t1) << " sec" << endl;
30     }

```

```

31     return 0;
32 }

```

Parallel Code (2D Threading with collapse)

```

1  #include <iostream>
2  #include <omp.h>
3  #include <cstdlib>
4  using namespace std;
5  #define N 1000
6  double A[N][N], B[N][N], C[N][N];
7  int main() {
8      double t1, t2;
9      int threads[] = {1, 2, 4, 8, 16};
10     srand(42);
11     // initialize matrices...
12     // 2D parallel (collapse)
13     for(int j = 0; j < 5; j++) {
14         int nt = threads[j];
15         omp_set_num_threads(nt);
16         t1 = omp_get_wtime();
17         #pragma omp parallel for collapse(2)
18         for(int i = 0; i < N; i++) {
19             for(int j = 0; j < N; j++) {
20                 C[i][j] = 0;
21                 for(int k = 0; k < N; k++) {
22                     C[i][j] += A[i][k] * B[k][j];
23                 }
24             }
25         }
26         t2 = omp_get_wtime();
27         cout << nt << " threads: " << (t2-t1) << " sec" << endl;
28     }
29     return 0;
30 }

```

Parallel Output:

```

1  Matrix Multiplication 1000x1000
2  =====
3  Running serial...
4  Serial time: 6.359 sec
5  === 1D Threading ===
6  Threads Time           Speedup
7  1         6.458         0.985x
8  2         3.994         1.592x
9  4         2.541         2.503x
10 8         2.153         2.954x
11 16        2.215         2.871x
12 === 2D Threading (collapse) ===
13 Threads Time           Speedup
14 1         6.862         0.927x
15 2         3.733         1.703x
16 4         2.442         2.604x
17 8         2.144         2.966x
18 16        2.179         2.918x

```

Answer

Comparison of 1D vs 2D Threading:

| Threads | 1D Speedup | 2D Speedup |

|-----|-----|-----|

| 1 | 0.985x | 0.927x |

| 2 | 1.592x | 1.703x |

| 4 | 2.503x | 2.604x |

| 8 | 2.954x | 2.966x |

| 16 | 2.871x | 2.918x |

Observations:

1. **Both approaches achieve similar speedup:** At 8 threads, 1D threading achieves 2.95x speedup while 2D threading achieves 2.97x speedup.
 2. **1D Threading advantages:**
 - Better cache locality - each thread processes complete rows
 - Row A[i] stays in cache while computing all C[i][j]
 - Less thread scheduling overhead
 3. **2D Threading (collapse) advantages:**
 - Better load balancing for irregular workloads
 - More parallelism available (N² iterations instead of N)
 - Useful when N is small relative to thread count
 4. **Cache behavior:** In 1D threading, thread i accesses row A[i] repeatedly, which stays in cache. In 2D threading, cache behavior can be less predictable.
 5. **Maximum speedup ~3x:** Despite having 8 cores, the maximum speedup is around 3x. This is because matrix multiplication has significant memory access overhead.
- Conclusion:** For this workload, 1D and 2D threading perform similarly. 1D threading is generally preferred for matrix multiplication due to better cache locality and simpler implementation.

Question 3

Q3. Calculation of π : This is the first example where many threads will cooperate to calculate a computational value. The task of this program will be arrive at an approximate value of π . The value of π is given by the following integral:

$$\pi = \int$$

4.0

1 + x

2

dx

1

0

This can be approximated as the sum of the areas of rectangles under the curve.

Sample pseudocode:

```
static long num_steps = 100000;
```

```
double step;
```

```
void main ()
```

```
{
```

```
int i;
```

```
double x, pi, sum = 0.0;
```

```
step = 1.0/(double)num_steps;
```

```
for (i=0; i<num_steps; i++) {
```

```
    x = (i+0.5)step;
```

```
sum = sum + 4.0/(1.0+xx);  
}  
pi = step * sum;  
}
```

Sequential Code

```
1  #include <iostream>  
2  #include <cmath>  
3  #include <omp.h>  
4  using namespace std;  
5  #define NUM_STEPS 100000000  
6  int main() {  
7      double step = 1.0 / NUM_STEPS;  
8      double pi, sum;  
9      double t1, t2;  
10     cout << "Calculating PI" << endl;  
11     cout << "Steps: " << NUM_STEPS << endl;  
12     // serial  
13     sum = 0.0;  
14     t1 = omp_get_wtime();  
15     for(long i = 0; i < NUM_STEPS; i++) {  
16         double x = (i + 0.5) * step;  
17         sum += 4.0 / (1.0 + x*x);  
18     }  
19     pi = step * sum;  
20     t2 = omp_get_wtime();  
21     cout << "PI = " << pi << endl;  
22     cout << "Time = " << (t2 - t1) << " sec" << endl;  
23     cout << "Error = " << fabs(pi - 3.14159265358979) << endl;  
24     return 0;  
25 }
```

Sequential Output:

```
1 Calculating PI  
2 Steps: 100000000  
3 Serial:  
4 PI = 3.14159  
5 Time = 0.323 sec  
6 Error = 6.3638e-13
```

Parallel Code (with different synchronization methods)

```
1  #include <iostream>  
2  #include <cmath>  
3  #include <omp.h>  
4  using namespace std;  
5  #define NUM_STEPS 100000000  
6  
7  int main() {  
8      double step = 1.0 / NUM_STEPS;  
9      double pi, sum;  
10     double t1, t2;  
11     omp_set_num_threads(4);  
12     // Method 1: Reduction
```

```

13     sum = 0.0;
14     t1 = omp_get_wtime();
15     #pragma omp parallel for reduction(+:sum)
16     for(long i = 0; i < NUM_STEPS; i++) {
17         double x = (i + 0.5) * step;
18         sum += 4.0 / (1.0 + x*x);
19     }
20     pi = step * sum;
21     t2 = omp_get_wtime();
22     cout << "Reduction: " << (t2-t1) << " sec" << endl;
23     // Method 2: Critical Section
24     sum = 0.0;
25     t1 = omp_get_wtime();
26     #pragma omp parallel
27     {
28         double local = 0;
29         #pragma omp for
30         for(long i = 0; i < NUM_STEPS; i++) {
31             double x = (i + 0.5) * step;
32             local += 4.0 / (1.0 + x*x);
33         }
34         #pragma omp critical
35         sum += local;
36     }
37     pi = step * sum;
38     t2 = omp_get_wtime();
39     cout << "Critical: " << (t2-t1) << " sec" << endl;
40     // Method 3: Atomic
41     sum = 0.0;
42     t1 = omp_get_wtime();
43     #pragma omp parallel
44     {
45         double local = 0;
46         #pragma omp for
47         for(long i = 0; i < NUM_STEPS; i++) {
48             double x = (i + 0.5) * step;
49             local += 4.0 / (1.0 + x*x);
50         }
51         #pragma omp atomic
52         sum += local;
53     }
54     pi = step * sum;
55     t2 = omp_get_wtime();
56     cout << "Atomic: " << (t2-t1) << " sec" << endl;
57     return 0;
58 }

```

Parallel Output:

```

1  Calculating PI
2  Steps: 100000000
3  =====
4  Serial:
5  PI = 3.14159
6  Time = 0.323 sec
7  Error = 6.3638e-13
8  Parallel (reduction):
9  Threads PI                Time      Speedup
10 1      3.14159            0.311    1.039x

```

11	2	3.14159	0.171	1.889x
12	4	3.14159	0.130	2.485x
13	8	3.14159	0.106	3.047x
14	16	3.14159	0.098	3.296x
15				
16	Comparing sync methods (4 threads):			
17	Reduction: 0.124 sec			
18	Critical: 0.115 sec			
19	Atomic: 0.107 sec			

Answer

Comparison of Synchronization Methods:

Method	Time (4 threads)	Description
Reduction	0.124 sec	OpenMP handles private copies and combines at end
Critical	0.115 sec	Only one thread enters critical section at a time
Atomic	0.107 sec	Hardware-level atomic operation for single update

Observations:

- All methods give correct result:** $\pi = 3.14159265358979$ with error $\sim 10^{-13}$
- Reduction clause:**
 - Cleanest syntax: `reduction(+:sum)`
 - OpenMP creates private copies for each thread
 - Combines all copies at the end of parallel region
 - Best for most use cases
- Critical section:**
 - Manually accumulate local sums to shared variable
 - Only one thread can enter critical section at a time
 - More overhead due to locking mechanism
 - Useful when reduction is not applicable
- Atomic operation:**
 - Similar to critical but for single memory operations
 - Uses hardware atomic instructions
 - Less overhead than critical for simple operations
 - Limited to specific operations (+, -, *, /, &, |, ^, etc.)
- Speedup analysis:**
 - Maximum speedup of 3.3x achieved with 16 threads
 - Near-linear speedup because this is compute-bound (not memory-bound)
 - Each iteration is independent, perfect for parallelization

Conclusion:

- Use `reduction` for cleaner code and good performance
- Use `atomic` for simple single-variable updates when reduction doesn't apply
- Use `critical` only when you need to protect a code block (multiple statements)
- All three methods are correct; choice depends on code structure and readability

Summary

Question	Operation	Best Speedup	Optimal Threads
----------	-----------	--------------	-----------------

Q1	DAXPY	1.18x	8
Q2	Matrix Multiply	2.97x	8

Key Learnings:

1. Memory-bound operations (DAXPY) show limited speedup
2. Compute-bound operations (Pi calculation) scale better with threads
3. Cache locality matters (1D vs 2D threading in matrix multiply)
4. Thread overhead can outweigh benefits for small workloads
5. Choose appropriate synchronization method based on use case